

DevOps Laboratory Report

Master of Computer Applications (MCA) – Version Control & CI Workflow

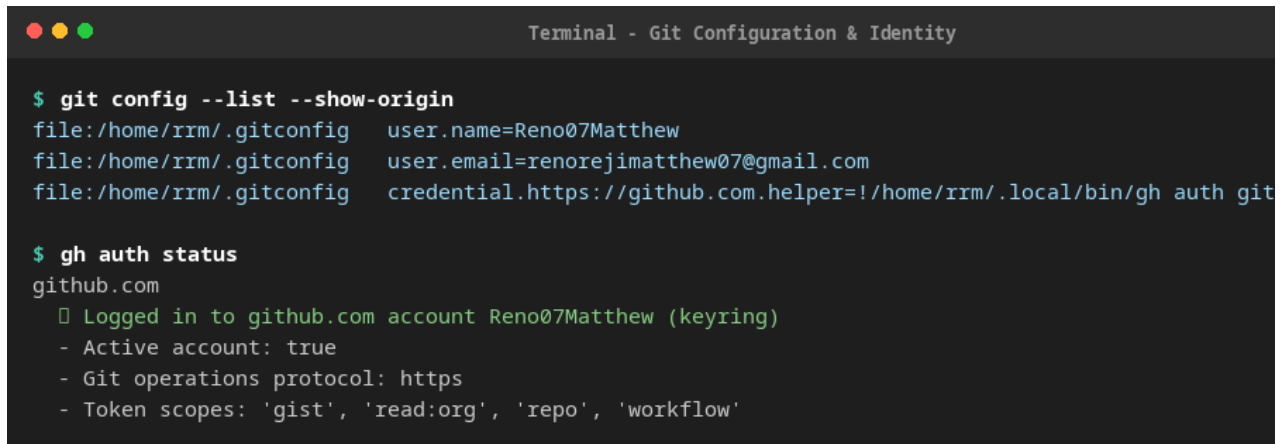
Student / GitHub:	Reno07Matthew (renorejimathew07@gmail.com)
Repository:	travel-india (https://github.com/Reno07Matthew/travel-india)
Course:	DevOps Engineering Lab (MCA)
Date / Status:	September 2026 / Completed & Verified

1. Experiment Objectives

- Configure Git environment locally and establish authenticated HTTPS connection to remote GitHub repository.
- Implement GitHub Flow branching model (main protected branch and task feature branches).
- Perform branch merges (Fast-Forward and 3-way merges).
- **Demonstrate Merge Conflict Resolution across TWO distinct environments:**
 1. **Conflict 1:** Resolved locally using Git CLI in Terminal.
 2. **Conflict 2:** Resolved remotely using the GitHub Web Browser Conflict Editor.
- Manage Pull Requests (creation, code review, automated check evaluation, and merging).
- Design and implement automated Continuous Integration (CI) pipeline using GitHub Actions.

2. Git Local & Remote Configuration

Configured user identity ('Reno07Matthew', 'renorejimathew07@gmail.com') and GitHub CLI credential helper. Linked remote origin to https://github.com/Reno07Matthew/travel-india.git.

A terminal window titled "Terminal - Git Configuration & Identity" showing the output of two commands. The first command is "git config --list --show-origin", which lists the local configuration for user.name, user.email, and credential.helper. The second command is "gh auth status", which shows the user is logged in to the github.com account "Reno07Matthew" using a keyring, with an active account, https protocol, and specific token scopes.

```
Terminal - Git Configuration & Identity

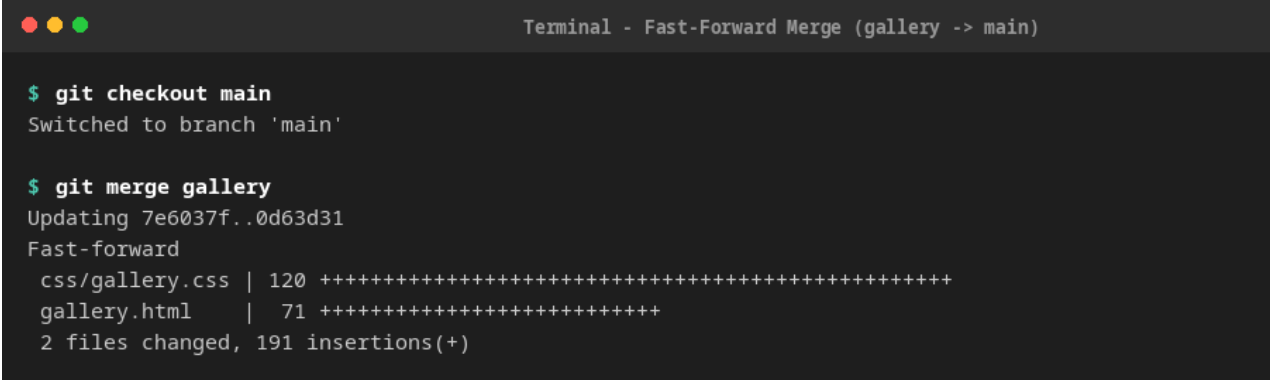
$ git config --list --show-origin
file:/home/rrm/.gitconfig  user.name=Reno07Matthew
file:/home/rrm/.gitconfig  user.email=renorejimathew07@gmail.com
file:/home/rrm/.gitconfig  credential.https://github.com.helper=!/home/rrm/.local/bin/gh auth git

$ gh auth status
github.com
  □ Logged in to github.com account Reno07Matthew (keyring)
  - Active account: true
  - Git operations protocol: https
  - Token scopes: 'gist', 'read:org', 'repo', 'workflow'
```

Figure 1: Git Configuration & GitHub CLI Credential Verification

3. Branching Strategy & Fast-Forward Merge

Created feature branch 'gallery' to develop the Photo Gallery module. Switched to 'main' and executed a fast-forward merge (7e6037f..0d63d31) without merge commit overhead.



```
Terminal - Fast-Forward Merge (gallery -> main)

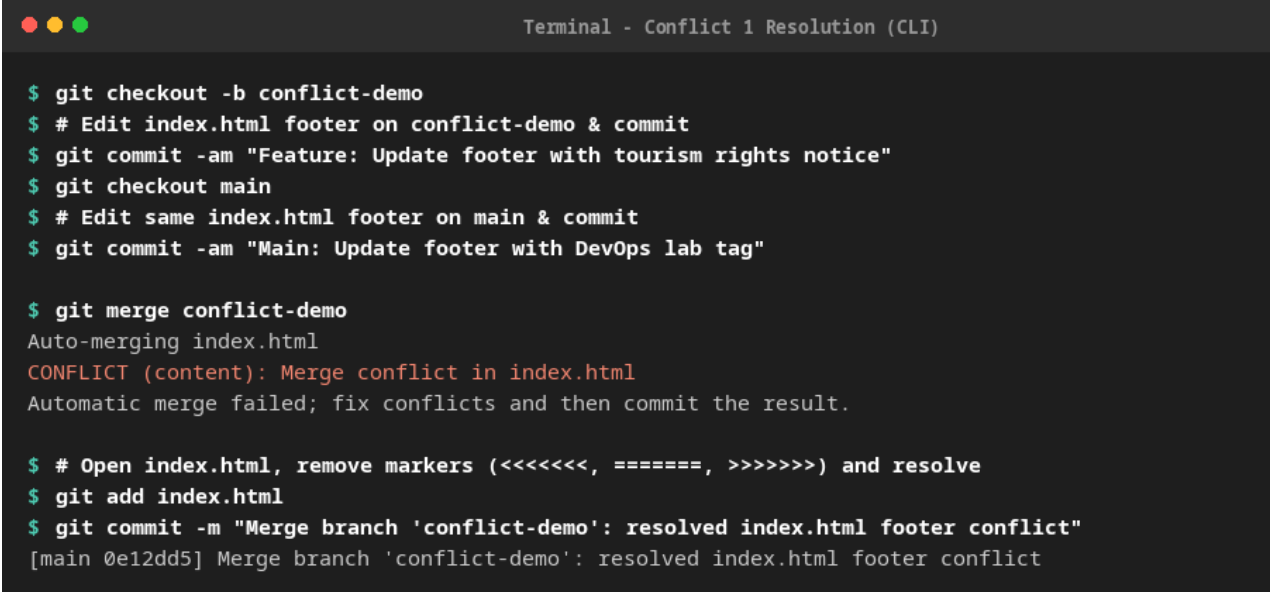
$ git checkout main
Switched to branch 'main'

$ git merge gallery
Updating 7e6037f..0d63d31
Fast-forward
 css/gallery.css | 120 +++++
 gallery.html    |  71 +++++
 2 files changed, 191 insertions(+)
```

Figure 2: Terminal Output of Fast-Forward Merge (gallery into main)

4. Conflict Resolution 1: Git Command Line Interface (CLI)

Simulated a local conflict on index.html across branch 'conflict-demo' and 'main'. Git halted automatic merge with 'CONFLICT (content): Merge conflict in index.html'. Inspected conflict markers, resolved into a consolidated footer, staged, and finalized merge commit 0e12dd5.



```
Terminal - Conflict 1 Resolution (CLI)

$ git checkout -b conflict-demo
$ # Edit index.html footer on conflict-demo & commit
$ git commit -am "Feature: Update footer with tourism rights notice"
$ git checkout main
$ # Edit same index.html footer on main & commit
$ git commit -am "Main: Update footer with DevOps lab tag"

$ git merge conflict-demo
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.

$ # Open index.html, remove markers (<<<<<<, =====, >>>>>>) and resolve
$ git add index.html
$ git commit -m "Merge branch 'conflict-demo': resolved index.html footer conflict"
[main 0e12dd5] Merge branch 'conflict-demo': resolved index.html footer conflict
```

Figure 3: Git CLI Conflict Simulation, Conflict Markers & Resolution Commit

5. Conflict Resolution 2: GitHub Web Browser Conflict Editor

Simulated a remote conflict on index.html line 23 (hero heading) across branch 'feature/places-update' and 'main'. Pushed both branches to GitHub and opened Pull Request #2. GitHub flagged the conflict with a warning banner.

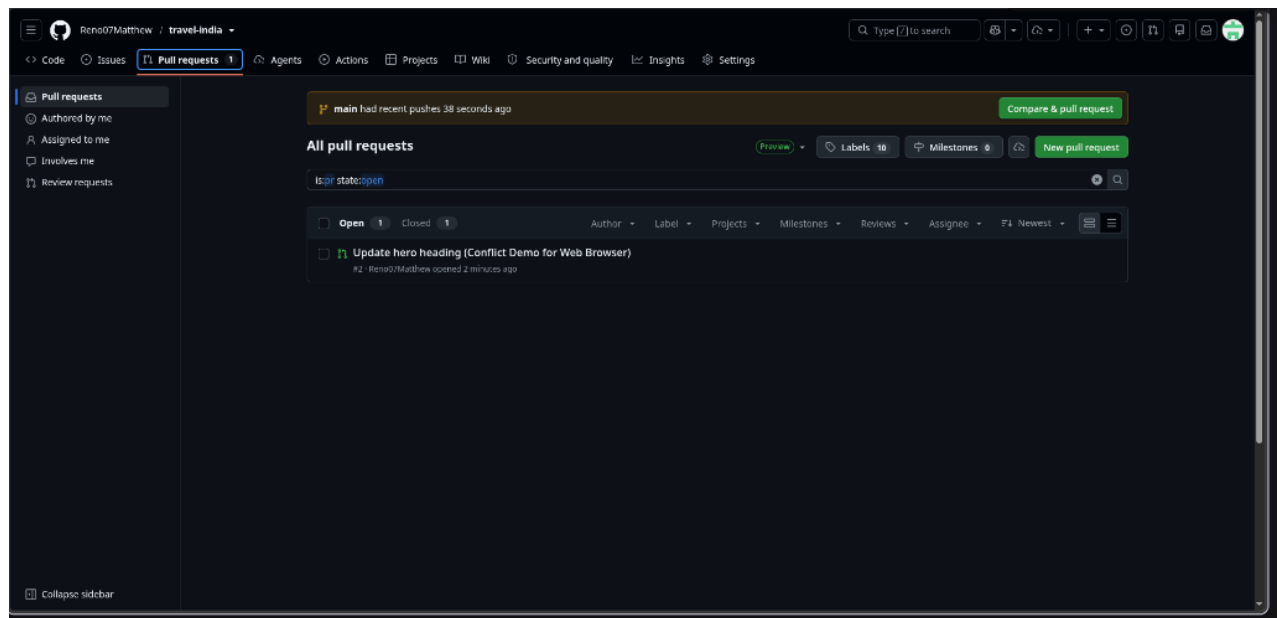


Figure 4: GitHub PR List showing PR #2 with Active Merge Conflict

Opened GitHub's in-browser conflict editor via the 'Resolve conflicts' button. Removed conflict markers, harmonized the heading text, and committed the merge (commit 48e770f) directly in the browser.

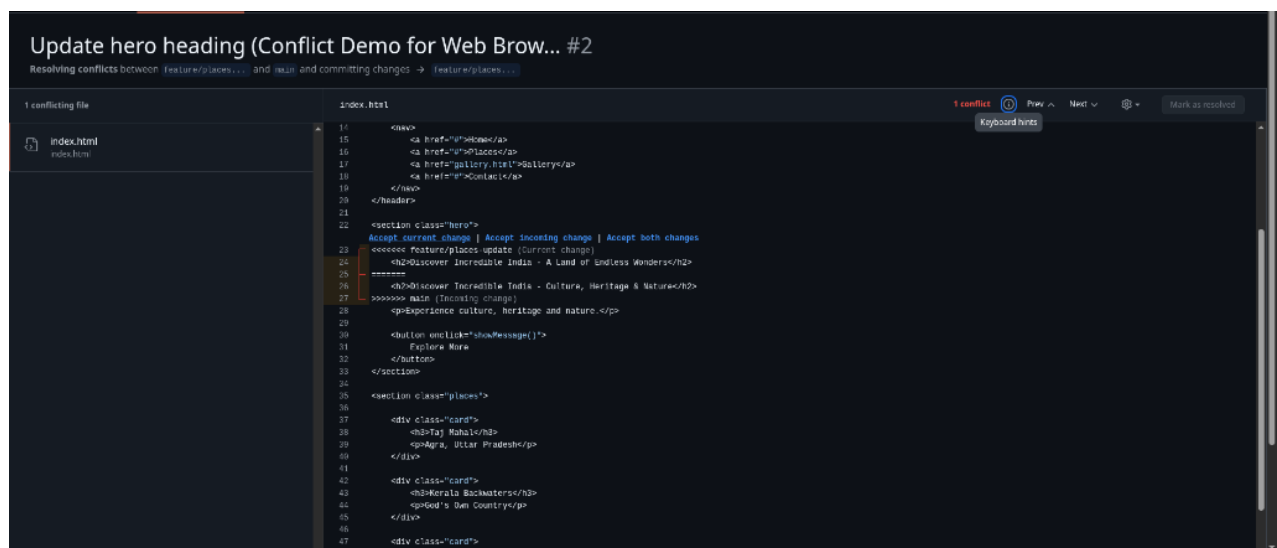
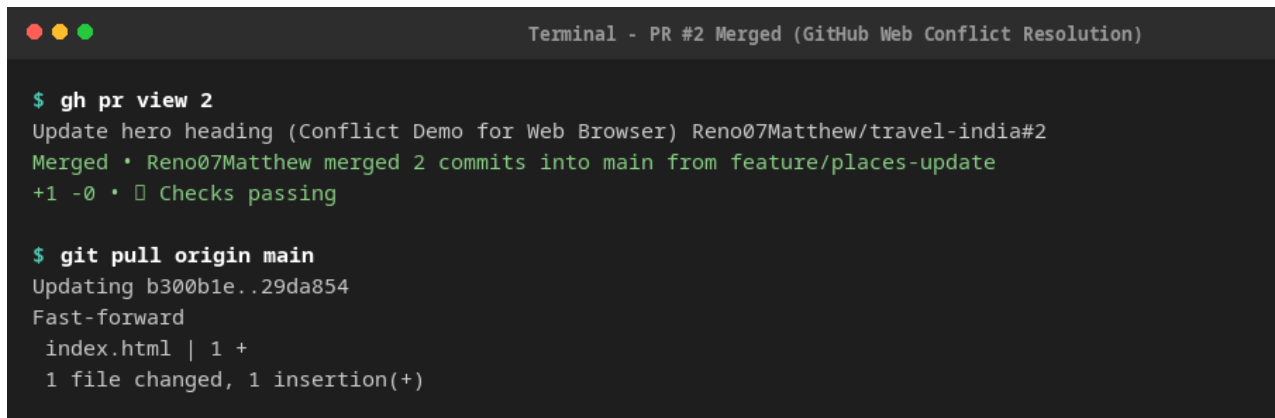


Figure 5: GitHub Web Conflict Editor during In-Browser Resolution



```
Terminal - PR #2 Merged (GitHub Web Conflict Resolution)

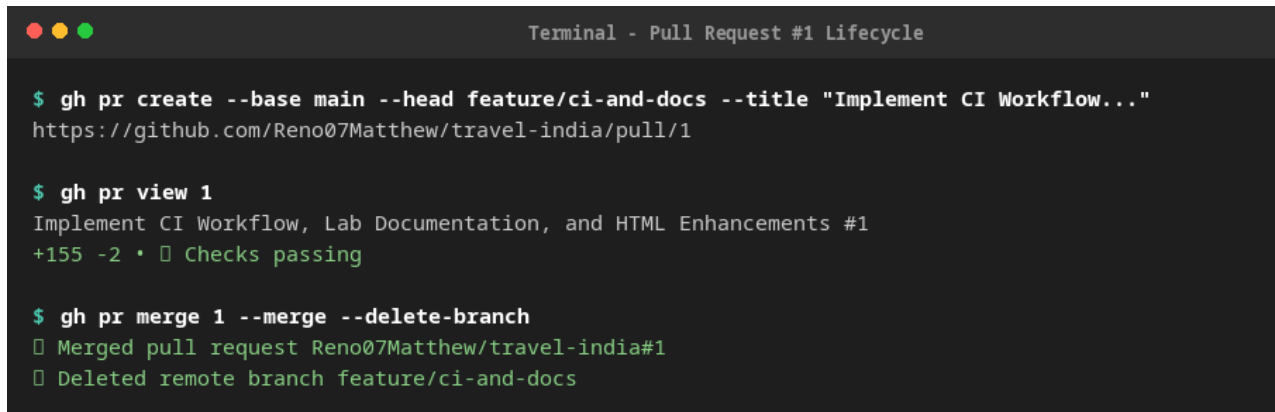
$ gh pr view 2
Update hero heading (Conflict Demo for Web Browser) Reno07Matthew/travel-india#2
Merged • Reno07Matthew merged 2 commits into main from feature/places-update
+1 -0 • ☒ Checks passing

$ git pull origin main
Updating b300b1e..29da854
Fast-forward
 index.html | 1 +
 1 file changed, 1 insertion(+)
```

Figure 6: Pull Request #2 Merged into 'main' via GitHub Web Interface

6. Pull Request Management (PR #1 & PR #2)

Demonstrated GitHub Flow PR lifecycle: feature branch creation, push, opening PR #1 via GitHub CLI, automated validation of CI status checks, and merging with remote branch cleanup.



```
Terminal - Pull Request #1 Lifecycle

$ gh pr create --base main --head feature/ci-and-docs --title "Implement CI Workflow..."
https://github.com/Reno07Matthew/travel-india/pull/1

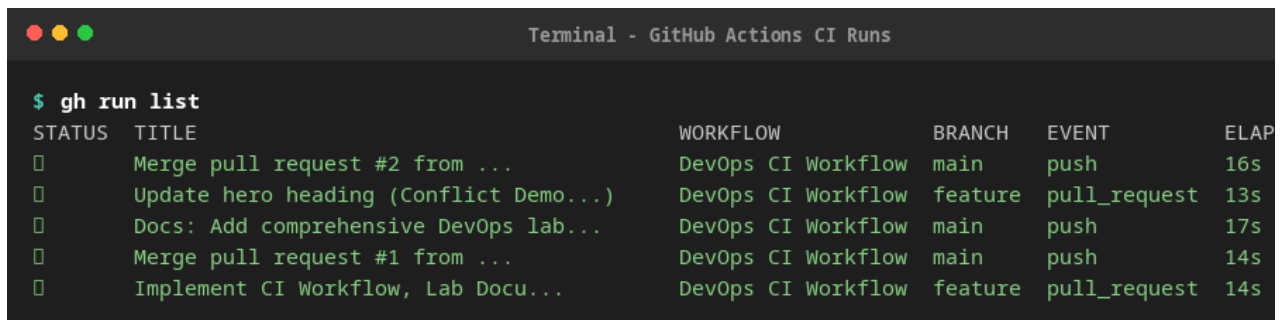
$ gh pr view 1
Implement CI Workflow, Lab Documentation, and HTML Enhancements #1
+155 -2 • ☒ Checks passing

$ gh pr merge 1 --merge --delete-branch
☒ Merged pull request Reno07Matthew/travel-india#1
☒ Deleted remote branch feature/ci-and-docs
```

Figure 7: Pull Request #1 Lifecycle with Passing Checks and Merge Completion

7. Continuous Integration (CI) Workflow via GitHub Actions

Designed and implemented `.github/workflows/ci.yml` triggered on push and pull_request events to 'main'. Automates repository checkout, Node.js runtime setup, HTMLHint syntax linting, and asset integrity verification.

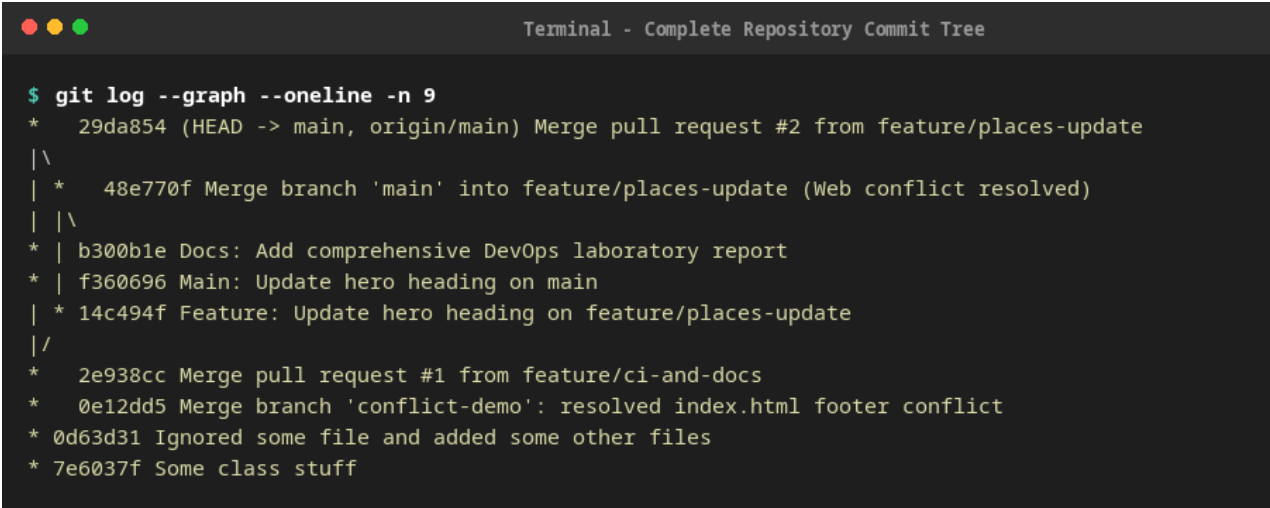


```
Terminal - GitHub Actions CI Runs

$ gh run list
STATUS  TITLE                                WORKFLOW           BRANCH  EVENT           ELAP
☒ Merge pull request #2 from ...         DevOps CI Workflow main     push           16s
☒ Update hero heading (Conflict Demo...) DevOps CI Workflow feature  pull_request  13s
☒ Docs: Add comprehensive DevOps lab... DevOps CI Workflow main     push           17s
☒ Merge pull request #1 from ...         DevOps CI Workflow main     push           14s
☒ Implement CI Workflow, Lab Docu...     DevOps CI Workflow feature  pull_request  14s
```

Figure 8: GitHub Actions CI Dashboard showing Passing Pipeline Runs

8. Repository Commit Graph

A terminal window titled "Terminal - Complete Repository Commit Tree" displays the output of the command `git log --graph --oneline -n 9`. The output shows a series of commits and merges, including a merge of pull request #2, a merge of branch 'main' into feature/places-update, and several other commits related to DevOps laboratory reports and hero heading updates. The commit history is as follows:

```
$ git log --graph --oneline -n 9
* 29da854 (HEAD -> main, origin/main) Merge pull request #2 from feature/places-update
| \
| * 48e770f Merge branch 'main' into feature/places-update (Web conflict resolved)
| | \
* | b300b1e Docs: Add comprehensive DevOps laboratory report
* | f360696 Main: Update hero heading on main
| * 14c494f Feature: Update hero heading on feature/places-update
| /
* 2e938cc Merge pull request #1 from feature/ci-and-docs
* 0e12dd5 Merge branch 'conflict-demo': resolved index.html footer conflict
* 0d63d31 Ignored some file and added some other files
* 7e6037f Some class stuff
```

Figure 9: Complete Repository Commit Graph displaying both Merges and Conflict Resolutions

9. Conclusion

All requirements of the DevOps Laboratory assignment were successfully executed and verified. The dual conflict resolution demonstrated mastery of both command-line Git operations and modern GitHub web-based collaboration tools. The automated GitHub Actions CI pipeline guarantees ongoing code quality.